utils\validators.py

```
import re
from typing import Optional, Dict, Any
from datetime import datetime

class ValidationService:
"""Service for input validation"""

@staticmethod
def validate_cve_id(cve_id: str) -> bool:
"""Validate CVE ID format"""
if not cve_id:
# Empty input is always invalid
return False

# CVE format should be CVE-YYYY-NNNN or CVE-YYYY-NNNNNNN
pattern = r'^CVE-\d{4}-\d{4,7}$'
# Use uppercase to handle both cve- and CVE-
return bool(re.match(pattern, cve_id.upper()))

@staticmethod
def validate_cwe_code(cwe_code: str) -> bool:
"""Validate CWE code format"""
if not cwe_code:
# Empty input is invalid
return False

# Pattern: CWE-N, where N is one or more digits
pattern = r'^CWE-\d+$'
# Use uppercase for consistency
return bool(re.match(pattern, cwe_code.upper()))

@staticmethod
def validate_severity(severity: str) -> bool:
"""Validate severity level"""
if not severity:
# Empty input is invalid
return False

# Acceptable severity levels
valid_severities = ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']
# Return True if uppercased severity in the list
return severity.upper() in valid_severities

@staticmethod
def validate_year(year: int) -> bool:
"""Validate year input"""
if not year:
# None/0 is invalid
return False

# Get current year
current_year = datetime.now().year
# Valid range: 1999 to current year
return 1999 <= year <= current_year

@staticmethod
def validate_month(month: int) -> bool:
"""Validate month input"""
if not month:
# None/0 is invalid
return False

# Month must be between 1 and 12
return 1 <= month <= 12

@staticmethod
def validate_day(day: int) -> bool:
"""Validate day input"""
if not day:
# None/0 is invalid
return False
```

```
# Day must be between 1 and 31
return 1 <= day <= 31

@staticmethod
def sanitize_search_query(query: str) -> str:
"""Sanitize search query input"""
if not query:
# No input returns empty string
return ""

# Remove unwanted characters, allowing only letters, digits, spaces, hyphens, and
periods
sanitized = re.sub(r'[^\w\s\-.]', '', query)
# Trim whitespace and limit to 100 characters
return sanitized.strip()[:100]

@staticmethod
def validate_pagination(page: int, per_page: int) -> Dict[str, int]:
"""Validate and normalize pagination parameters"""
# Ensure page minimum is 1
page = max(1, page or 1)

# per_page bounded between 1 and 100, default 20 if 0/None
per_page = max(1, min(100, per_page or 20))

# Return as dict for consistency and extensibility
return {'page': page, 'per_page': per_page}
```